`do` expressions

Document #:	P2806R2
Date:	2023-11-16
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Bruno Cardoso Lopes <bruno.cardoso@gmail.com> Zach Laine <whatwasthataddress@gmail.com> Michael Park <mcypark@gmail.com> Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2806R1], switched syntax from `do return` to `do_return` to avoid ambiguity. Added section on lifetime.

Since [P2806R0], some more discussion about implicit last value vs explicit return, reflection, and a grammar fix to the still-incomplete wording.

§ 2 Introduction

C++ is a language built on statements. `if` is not an expression, loops aren't expressions, statements aren't expressions (except maybe in the specific case of *`expression;`*).

When a single expression is insufficient, the only solution C++ currently has as its disposal is to invoke a function - where that function can now contain arbitrarily many statements. Since C++11, that function can be expressed more conveniently in the form of an immediately invoked lambda.

However, this approach leaves a lot to be desired. An immediately invoked lambda introduced an extra function scope, which makes control flow much more challenging - it becomes impossible to `break` or `continue` out of a loop, and attempting to `return` from the enclosing function or `co_await`, `co_yield`, or `co_return` from the enclosing coroutine becomes an exercise in cleverness.

You also have to deal with the issue that the difference between initializing a variable used an immediately-invoked lambda and initializing a variable from a lambda only differs in the trailing `()`, arbitrarily deep into an expression, which are easy to forget. Some people actually use `std::invoke` in this context, specifically to make it clearer that this lambda is, indeed, intended to be immediately invoked.

This problem surfaces especially brightly in the context of pattern matching [P1371R3], where the current design is built upon a sequence of:

```
pattern => expression;
```

This syntax only allows for a single *`expression`*, which means that pattern matching has to figure out how to deal with the situation where the user wants to write more than, well, a single expression. The current design is to allow `{ `*`statement`*` }` to be evaluated as an expression of type `void`. This is a hack, which is kind of weird (since such a thing is not actually an expression of type `void`), but also limits the ability for pattern matching to support another kind of useful syntax: `=>` *`braced-init-list`*:

```
auto f() -> std::pair<int, int> {
    // this is fine
    return {1, 2};

    // this is ill-formed in P1371
    return true match -> std::pair<int, int> {
        _ => {1, 2}
    };
}
```

There's no way to make that work, because `{` starts a statement. So the choice in that paper lacks orthogonality: we have a hack to support multiple expressions (which are very important to support) that is inventing such support on the fly, in a novel way that is very narrow (only supports `void`), that throws other useful syntax under the bus.

What pattern matching really needs here is a statement-expression syntax. But it's not just pattern matching that has a strong desire for statement-expressions, this would be a broadly useful facility, so we should have an orthogonal language feature that supports statement-expressions in a way that would allow pattern matching to simplify its grammar to:

```
pattern => expr-or-braced-init-list;
```

3 `do` expressions

Our proposal is the addition of a new kind of expression, called a `do` expression.

In its simplest form:

```
int x = do { do_return 42; };
```

A `do` expression consists of a sequence of statements, but is still, itself, an expression (and thus has a value and a type). There are a lot of interesting rules that we need to discuss about how those statements behave.

3.1 Scope

A `do` expression does introduce a new block scope - as the braces might suggest. But it does *not* introduce a new function scope. There is no new stack frame. Which is what allows external control flow to work (see below).

3.2 `do_return` statement

The new `do_return` statement has the same form as the `return` statement we have today: `do_return` *`expr-or-braced-init-list`*$_{\texttt{opt}}$`;`. It's behavior corresponds closely to that `return`, in unsurprising ways - `do_return` yields from a `do` expression in the same way that `return` returns from a function.

While `do_return` *`value`*`;` and `return` *`value`*`;` do look quite close together and mean fairly different things, the leading `do` we think should be sufficiently clear, and we think it is a good spelling for this statement.

Other alternative spellings we've considered:

- `do return` (in the previous revision of this paper, which has an ambiguity with `do ... while` loops)
- `do_yield` (presented to EWG in Issaquah as the initial pre-publication draft of this proposal)
- `do yield`
- `do break` (similarly to `return`, we are breaking out of this expression, but is less likely to conflict since `break` is less likely to be used than `return` and also the corresponding `break` *`value`*`;` is invalid today)
- `=>` (or some other arrow, like `<-` or `<=`)

§ 3.3 Type and Value Category

The expression `do { do_return 42; }` is a prvalue of type `int`. We deduce the type from all of the `do_return` statements, in the same way that `auto` return type deduction works for functions and lambdas.

An explicit *`trailing-return-type`* can be provided to override this:

```
do -> long { do_return 42; }
```

If no `do_return` statement appears in the body of the `do` expression, or every `do_return` statement is of the form `do_return;`, then the expression is a prvalue of type `void`.

Falling off the end of a `do` expression behaves like an implicit `do_return;` - if this is incompatible the type of the `do` expression, the expression is ill-formed. This is the one key difference with functions: this case is not undefined behavior. This will be discussed in more detail later.

This makes the pattern matching cases [P2688R0] work pretty naturally:

P2688R0	Proposed

`x match {`
`  0 => { cout << "got zero"; };`
`  1 => { cout << "got one"; };`
`  _ => { cout << "don't care"; };`
`}`
	`x match {`
	`  0 => do { cout << "got zero"; };`
	`  1 => do { cout << "got one"; };`
	`  _ => do { cout << "don't care"; };`
	`}`

Here, the whole `match` expression has type `void` because each arm has type `void` because none of the `do` expressions have a `do_return` statement.

Yes, this requires an extra `do` for each arm, but it means we have a language that's much easier to explain because it's consistent - `do { cout << "don't care"; }` is a `void` expression in *any* context. We don't have a *`compound-statement`* that happens to be a `void` expression just in this one spot.

P2688R0	Proposed

`auto f(int i) {`
`  return i match -> std::pair<int,`
`  int> {`
`  0 => {1, 2};  // ill-formed`
`  _ => std::pair{3, 4}; // ok`
`  }`
`}`
	`auto f(int i) {`
	`  return i match -> std::pair<int,`
	`  int> {`
	`  0 => {1, 2};  // ok`
	`  _ => std::pair{3, 4}; // ok`
	`  }`
	`}`

Here, the existing pattern matching cannot support a *`braced-init-list`* because `{` is used for the special `void`-statement-case. But if we had `do` expressions, the grammar of pattern matching can use *`expr-or-braced-init-list`* in the same way that we already do in many other places in the C++ grammar. This example just works.

§ 3.4 Copy Elision

All the rules for initializing from `do` expression, and the way the expression that appears in a `do_return` statement is treated, are the same as what the rules are for `return`.

Implicit move applies, for variables declared within the body of the `do` expression. In the following example, `r` is an unparenthesized *`id-expression`* that names an automatic storage variable declared within the statement, so it's implicitly moved:

```
std::string s = do {
    std::string r = "hello";
    r += "world";
    do_return r;
};
```

Note that automatic storage variables declared within the function that the `do` expression appears, but not declared within the statement-expression itself, are *not* implicitly moved (since they can be used later).

§ 3.5 Control Flow

In a regular function, there are four ways to escape the function scope:

1. a `return` statement
2. `throw`ing an exception
3. invoking a `[[noreturn]]` function, `std::abort()` and `std::unreachable()`
4. falling off the end of the function (undefined behavior if the return type is not `void`)

The same is true for coroutines, except substituting `return` for `co_return` (and likewise falling off the end is undefined behavior if there is no `return_void()` function on the promise type).

For a `do` expression, we have two different directions where we can escape (in a non-exception, non-`[[noreturn]]` case): we either yield an expression, or we escape the *outer* scope. That is, we can also:

1. `return` from the enclosing function (or `co_return` from the enclosing coroutine)
2. `break` or `continue` from the innermost enclosing loop (if any, ill-formed otherwise)

Additionally, for point (4) while we could simply (for consistency) propagate the same rules for falling-off-the-end as functions, then lambdas (C++11), then coroutines (C++20), we would like to consider not introducing another case for undefined behavior here and enforcing that the user provides more information themselves.

That is, the rule we propose that the implementation form a control flow graph of the `do` expression and consider each one of the six escaping kinds described above. All `do_return` statements (including the implicit `do_return;` introduced by falling off the end, if the implementation cannot prove that it does not happen) need to either have the same type (if no *`trailing-return-type`*) or be compatible with the provided return type (if provided). Anything else is ill-formed.

Let’s go through some examples.

Example	Discussion

`auto a = do {`
`  if (cond) {`
`  do_return 1;`
`  } else {`
`  do_return 2;`
`  }`
`};`
	OK: All yielding control paths have the same type. There’s no falling off the end.

`auto b = do {`
`  if (cond) {`
`  do_return 1;`
`  } else {`
`  do_return 2.0;`
`  }`
`};`
	Ill-formed: The yielding control paths have different types and there is no provided *`trailing-return-type`*. This would be okay if it were `do -> int { ... }` or `do -> double { ... }` or `do -> float { ... }`, etc.

`auto c = do {`
`  if (cond) {`
`  do_return 1;`
`  }`
 	
`  do_return 2;`
`};`
	OK: Similar to a, all yielding control paths yield the same type. There is no falling off the end here, it is not important that a yielding `if` has an `else`.

`auto d = do {`
`  if (cond) {`
`  do_return 1;`
`  }`
`};`
	Ill-formed: There are two yielding control paths here: the `do_return 1;` and the implicit `do_return;` from falling off the end, those types are incompatible. The equivalent in functions and coroutines would be undefined behavior in if *`cond`* is `false`.

`int e = do {`
`  if (cond) {`
`  do_return;`
`  }`
`}, 1;`
	OK: As above, there are two yielding control paths here, but both the explicit and the implicit ones are `do_return;` which are compatible.

`int f = do {`
`  if (cond) {`
`  do_return 1;`
`  }`
 	
`  throw 2;`
`};`
	OK: We no longer fall off the end here, since we always escape. There is only one yielding path.

`int outer() {`
`  int g = do {`
`  if (cond) {`
`  do_return 1;`
`  }`
 	
`  return 3;`
`  };`
`}`
	OK: Similar to the above, it's just that we're escaping by returning from the outer function instead of throwing. Still not falling off the end.

`int h = do {`
`  if (cond) {`
`  do_return 1;`
`  }`
 	
`  std::abort();`
`};`
	OK: `std::abort()` means that we cannot fall off the end, see discussion on `[[noreturn]]` below.

`enum Color {`
`  Red,`
`  Green,`
`  Blue`
`};`
 	
`void func(Color c) {`
`  std::string_view name`
`  = do {`
`  switch (c) {`
`  case Red:`
`  do_return`
`  "Red"sv;`
`  case Green:`
`  do_return`
`  "Green"sv;`
`  case Blue:`
`  do_return`
`  "Blue"sv;`
`  }`
`  };`
`}`
	Ill-formed: This is probably the most interesting case when it comes to falling off the end. Here, the user knows that `c` only has three values, but the implementation does not, so it could still fall off the end. gcc does warn on the equivalent function form of this, clang does not. The typical solution here might be to add `__builtin_unreachable()`, now `std::unreachable()`, to the end of the function, but for this to work we have to discuss `[[noreturn]]` below. Barring that, the user would have to add either some default value or some other kind of control flow (like an exception, etc).

`void func() {`
`  for (;;) {`
`  int j = do {`
`  if (cond) {`
`  break;`
`  }`
 	
`  for`
`  (something) {`
`  if (cond)`
`  {`
	OK: The first `break` escapes the `do` expression and breaks from the outer loop. Otherwise, we have two yielding statements which both yield `int`. If the `do_return 2;` statement did not exist, this would be ill-formed unless the compiler could prove that the loop itself did not terminate.
	
	If the loop were `for (;;)`, then the lack of `do_return 2;` would be fine - but anything more complicated than that would require some kind of final yield (or `throw`, etc.)

```
        do_return 1;
                }
            }

            do_return 2;
        };
    }
}
```

To reiterate: the implementation produces a control flow graph of the `do` expression and considers all yielding statements (*including* the implicit `do_return;` on falling off the end, if the implementation considers that to be a possible path) in order to determine correctness of the statement-expression. The kinds of control flow that escape the statement entirely (exceptions, `return`, `break`, `continue`, `co_return`) do not need to be considered for purposes of consistency of yields (since they do not yield values).

§ 3.5.1 `noreturn` functions

The language currently has several kinds of escaping control flow that it recognizes. As mentioned, exceptions, `return`, `continue`, `break`, and `co_return`. And, allegedly, `goto`.

But there's one kind of escaping control flow that it *does not* currently recognize: functions marked `[[noreturn]]`. A call to `std::abort()` or `std::terminate()` or `std::unreachable()` escapes control flow, for sure, but today this is just an attribute:

```
int i = do {
    if (cond) {
        do_return 5;
    }

    std::abort();

    // we know control flow never gets here, so we should not need to
    // insert an implicit "do_return;"
};
```

Pattern Matching has this same problem - it needs to support arms that might `std::terminate()` or are `std::unreachable()`, so it that proposal currently is introducing a dedicated syntax to mark an arm as non-returning: `!{ std::terminate(); }`. Which is… less than ideal.

However, the rule in 9.12.10 [dcl.attr.noreturn]/2 is:

> 2 If a function `f` is called where `f` was previously declared with the `noreturn` attribute and `f` eventually returns, the behavior is undefined.

That is normative wording which we can rely on. The above `do` expression can only fall off the end if `std::abort` returns, which is *already* undefined behavior. We can avoid introducing any new undefined behavior ourselves as part of this feature.

That is: invoking a function marked `[[noreturn]]` can be considering an escaping control flow in exactly the same way that `return`, `break`, `throw`, etc., are already.

Note that this violates the so-called Second Ignorability Rule suggested in [P2552R2], which is a great reason to ignore that rule.

3.5.2 Always-escaping expressions

Consider:

```
int i = do -> int {
    throw 42;
};
```

This is weird, but might end up as a result of template instantiation where maybe other control paths (guarded with an `if constexpr`) actually had `do_return` statements in them. So it needs to be allowed.

It does lead to an interesting question: what is `decltype(do { return; })`? We already define `decltype(throw 42)` to be `void`, so would this also be `void`? It's kind of an odd choice, and it would be nice if we had a specific type for an escaping expression. While we could come up with the right language facility to allow the conditional operator (`?:`) and pattern matching to work correctly by ignoring arms that are always-escaping, user-defined code would have no way of differentiating between a real void expression (`std::print("Hello {}!", "EWG")` is actually an expression of type `void`) and an artificial one (`std::abort()` is not really the same thing).

We could instead introduce a new type, `std::noreturn_t` (as an easier-to-type spelling of ⊥), change `decltype(throw e)` to be `std::noreturn_t` (since nobody actually writes this - code search results are exclusively in compiler test suites) and treat the return types of `[[noreturn]]` functions as `std::noreturn_t`. Then the type system gains understanding of always-escaping expressions/statements and the rules for pattern matching, the conditional operator, `do` expressions, and arbitrary user-defined libraries just fall out.

See reflector discussion here.

3.5.3 `goto`

Using `goto` in a `do` expression has some unique problems.

Jumping *within* a `do` expression should follow whatever restrictions we already have (see 8.8 [stmt.dcl]). Jumping *into* a `do` expression should be completely disallowed (we would call the *`statement`* of a `do` expression a control-flow limited statement).

Jumping *out* of a `do` expression is potentially useful though, in the same way that `break`, `continue`, and `return` are:

```
    for (loop_1) {
        for (loop_2) {
            int i = do {
                if (cond) {
```

```
                    goto done;
                }

                do_return value;
            };
        }
    }
done:
```

Breaking out of multiple loops is one of the uses of `goto` that has no real substitute today. The above example should be fine. But referring to any label that is in scope of the variable we're initializing needs to be disallowed - since we wouldn't have actually initialized the variable. We need to ensure that the 8.8 [stmt.dcl] rule is extended to cover this case.

Also, while computed goto is not a standard C++ feature, it would be nice to disallow this example, courtesy of (of course) JF Bastien (in this case, we are referring to a label that is within `v`'s scope. We're not jumping to it directly, but the ability to jump to it indirectly is still problematic):

```
#include <stdio.h>

struct label {
    static inline void* e;
    int v;

    label()
    try
        : v(({
            fprintf(stderr, "oh\n");
            e = &&awesome;
            throw 1;
            42;
        }))
    {
        fprintf(stderr, "no\n");
        awesome:
        fprintf(stderr, "you\n");
    } catch(...) {
        fprintf(stderr, "don't\n");
        goto *e;
    }
};

int main() {
    label l;
}
```

§ 3.5.4 Should falling off the end be undefined behavior?

Consider:

```
int i = do {
    if (cond) {
        do_return 0;
```

```
    }
};
```

Is this statement ill-formed (because there is a control path that falls off the end of the `do` expression, as discussed in this section) or should this statement be undefined behavior? The latter would be consistent with functions, lambdas, and coroutines (and not a if-you-squint-enough kind of consistency either, this would be exactly identical).

It would make for a simpler design if we adopted undefined behavior here, but we think it's a better design to force the user to cover all control paths themselves.

§ 3.6 Lifetime

One important question is: when are local variables declared within a `do` expression destroyed?

Consider the following:

```
auto f() -> std::expected<T, E>;
auto g(T) -> U;

auto h() -> std::expected<U, E> {
    auto u = g(do -> TYPE {
        auto result = f();
        if (not result) {
            return std::unexpected(std::move(result).error());
        }
        do_return *std::move(result);
    });
    return u;
}
```

The use of *`TYPE`* above is meant as a placeholder for either `T` or `T&&`, as we'll explain shortly.

What is the lifetime of the local variable `result`? There are three possible choices to this question:

1. It is destroyed at the next `}`. This is the most consistent choice with everything else in the language.
2. It is destroyed at the end of the statement in which it is appears (i.e. at the end of the full initialization of `u`). In other words, at the end of the *full-expression* (the *real* full-expression, not the nested *full-expression*s inside of the `do` expression).
3. It behaves as if it has local scope of the surrounding scope and is destroyed at the end of that outer scope, which in this case would be the end of `h()`.

The consequence of (1) is that `result` is destroyed before we enter the call to `g`. This means that if the `do` expression returned a reference (i.e. *`TYPE`* was `T&&`), that reference would immediately dangle. We would have to yield a `T`. This loses us some efficiency, since ideally both the `do` expression and `g` could just take a `T` - but now we have to incur a move.

The consequence of (2) is that we *can* yield a `T&&` because `result` isn't going to be destroyed yet, and we don't have a dangling reference. Unless we rewrite it this way:

```
auto f() -> std::expected<T, E>;
auto g(T&&) -> U;

auto h() -> std::expected<U, E> {
    T&& t = do -> T&& {
        auto result = f();
        if (not result) {
            return std::unexpected(std::move(result).error());
        }
        do_return *std::move(result);
    };
    return g(std::move(t));
}
```

With (1), in order to avoid dangling, the `do` expression must return a `T` (`t` can still be an rvalue reference, it would just bind to a temporary). With (2), we can allow the `do` expression to return `T&&`, but `t` would to be a `T` and not a `T&&`, since `result` is going to be destroyed at the end of the statement. We still incur a move, just slightly later.

Only with (3) - delaying destroying `result` until the closing of the innermost non-`do`-expression scope - is the above valid code that does not lead to any dangling reference.

Note that the above example is specifically mentioned in [P2561R2]'s section on lifetimes, where it is quite valuable that the equivalent sugared version does not dangle:

```
auto f() -> std::expected<T, E>;
auto g(T&&) -> U;

auto h() -> std::expected<U, E> {
    T&& t = f().try?;
    return g(std::move(t));
}
```

Choosing (3) allows the control flow operator proposal to be simply a lowering into a `do` expression.

On the other hand, consider this example:

```
int i = do {
        std::lock_guard _(mtx);
        do_return get(0);
    } + do {
        std::lock_guard _(mtx);
        do_return get(1);
    };
```

Each `do` expression is locking the same mutex, `mtx`. With (1), the two `lock_guards` are each destroyed at their nearest `}`, so the result of this code is that we lock the mutex, call `get(0)`, unlock the mutex, then lock the mutex, call `get(1)`, and unlock the mutex (or possibly the second `do` expression is evaluated first, doesn't matter). Following the usual C++ lifetime rules, most people would expect this to be valid code.

With either of the two approaches to extending lifetime, either (2) or (3), this deadlocks. The two `lock_guard`s aren't destroyed until after the initialization of `i`, or even later, and so whichever one is locked first is still alive when the second one is locked.

That means the choice is between extending the lifetime of variables to avoid dangling references and keeping the lifetime of variables the same to maintain the usual C++ destructor rules. The familiarity and expectation of the latter is so strong that (1) is likely the only option. After all, scoped lifetimes is one of the fundamental rules of C++.

This does suggest that there needs to be some way to explicitly extend a variable to the outer scope. After all, a `do` expression's control flow behaves as if its in that outer scope (we `return` from the enclosing function, `continue` the enclosing loop, etc.), so there is definitely a compelling argument to me made that variables belong in that scope as well. But they probably need some sort of annotation - something like a reverse lambda capture:

```
auto f() -> std::expected<T, E>;
auto g(T&&) -> U;

auto h() -> std::expected<U, E> {
    // The "anti-capture" of result means that it's actually declared in the
    // outer scope, as if before the variable t. If no such variable result
        in
    // the do expression's scope, the expression is ill-formed.
    T&& t = do [result] -> T&& {
        auto result = f();
        if (not result) {
            return std::unexpected(std::move(result).error());
        }
        do_return *std::move(result);
    };
    return g(std::move(t));
}
```

3.6.1 Conditional Lifetime Extension

An interesting sub-question on lifetimes is what does this do:

```
auto prvalue() -> T;

auto f() -> void {
    // lifetime extension, reference bound to temporary
    T const& r1 = prvalue();

    // dangling
    T const& r2 = []() -> T const& { return prvalue(); };

    // lifetime extension??
    T const& r3 = do -> T const& { do_return prvalue(); };
}
```

`r1` is our familiar lifetime extension case - a reference is bound to a temporary. `r2` is definitely dangling. The temporary is destroyed and definitely does not last as long as `r2`. But what about `r3`, is it more like

`r1` or `r2`?

In this case, we see the entire `do` expression - so unlike the general callable case, it may actually be possible for lifetime extension to work here.

But as we're thinking about it, let's make the example slightly more complicated:

```
auto lvalue() -> T const&;
auto prvalue() -> T;

auto g(bool c) -> void {
    T const& r4 = do -> T const& {
        if (c) {
            do_return lvalue();
        } else {
            do_return prvalue();
        }
    }

    T const& r5 = do -> T const& {
        if (c) {
            do_return lvalue();
        } else {
            T x = prvalue();
            do_return x;
        }
    }
}
```

If `r3` doesn't dangle (and we do lifetime extension), does `r4`? Well, presumably. But here we have a form of runtime-conditional lifetime extension. That's still seemingly doable - we would effectively have an `optional<T> __storage` that is declared before `r4` and then `r4` is either a reference into that or whatever we got from `lvalue()`.

But even if we made `r4` work, `r5` now almost certainly cannot - now we're definitely not binding a temporary to a reference, this is quite adrift from our usual rules.

Which makes us wonder if there's really any value in being adventurous here - and instead probably consider that there is no lifetime extension in any of these cases, not in `r3`, not in `r4`, and definitely not in `r5`.

§ 3.7 Grammar Disambiguation

We have to disambiguate between a `do` expression and a `do-while` loop.

In an expression-only context, the latter isn't possible, so we're fine there.

In a statement context, a `do` expression is completely pointless - you can just write statements. So we disambiguate in favor of the `do-while` loop. If somebody really, for some reason, wants to write a `do` expression statement, they can parenthesize it: `(do { do_return 42; });`. A statement that begins with a `(` has to then be an expression, so we're now in an expression-only context.

A previous iteration of the paper used `do return` (with a space), which would lead to an ambiguity in the following in the context of a `do` expression:

```
do return value; while (cond);
```

This could have been parsed as a `do return` statement followed by an infinite loop (that would never be executed because we've already returned out of the expression) or as a `do`-`while` loop containing a single, unbraced, return statement. If we use the `do_return` spelling, there's no such ambiguity, since the above can only be a `do ... while` loop.

Also because we would unconditionally parse a statement as beginning with `do` as a `do`-`while` loop, code like this would not work:

```
do { do_return X{}; }.foo();
```

Such code would also have to be parenthesized to disambiguate, which doesn't seem like a huge burden on the user.

§ 3.8 Prior Art

GCC has had an extension called statement-expressions for decades, which look very similar to what we're proposing here:

gcc	Proposed

({
  int y = foo();
  int z;
  if (y > 0) z = y;
  else z = -y;
  z;
})
	do {
	  int y = foo();
	  if (y > 0) {
	  do_return y;
	  } else {
	  do_return -y;
	  }
	}

The reason we're not simply proposing to standardize the existing extension is that there are two features we see that are lacking in it that are not easy to add:

1. The ability to specify a return type, which is critical for allowing statement-expressions to be lvalues.
2. The ability to support yielding out of different branches of `if`, due the implicit nature of the yield.

For (1), there is simply no obvious place to put the *`trailing-return-type`*. For (2), you can't turn `if`s into expressions in any meaningful way. It is fairly straightforward to answer both questions for our proposed form.

Let's also take the example motivating case from [P2561R2] and compare implicit last expression to explicit return:

Implicit Last Value	Explicit Return

`auto foo(int i) -> std::expected<int, E>`
 	
`auto bar(int i) -> std::expected<int, E> {`
`  int j = do {`
`  auto r = foo(i);`
`  if (not r) {`
`  return std::unexpected(r.error());`
`  }`
`  *r // <== NB: no semicolon`
`  };`
 	
`  return j * j;`
`}`
	`auto foo(int i) -> std::expected<int, E>`
	
	`auto bar(int i) -> std::expected<int, E> {`
	`  int j = do {`
	`  auto r = foo(i);`
	`  if (not r) {`
	`  return std::unexpected(r.error());`
	`  }`
	`  do_return *r;`
	`  };`
	
	`  return j * j;`
	`}`

In the simple cases, explicit last value (on the left) will be shorter than an explicit return (on the right). But implicit last value is more limited. We cannot do early return (by design), which means that a `do` expression would not be able to return from a loop either. We would have to extend the language to support `if` expressions, so that at the very least the first example above could be made easier - which would add more complexity to the design.

There's also the question of `void` expressions - which are where many of the pattern matching examples come from. In Rust, for instance, there is a differentiation based on the presence of a semicolon:

```
let a: i32 = { 1; 2 };
let b: () = { 1; 2; };
```

This is a simple (if silly) example of a block expression in Rust. The value of the block is the value of the last expression of the block (Rust has both `if` expressions and `loop` expressions) - in the first case the last example is `2`, so `a` is an `i32`, while in the second example `2;` is a statement, so the last value is the… nothing… after the `;`, which is `()` (Rust's unit type). This seems like too subtle a distinction, and one that's very easy to get wrong (although typically the types are far enough apart such that if you get it wrong it's a compiler error, rather than a runtime one):

```
auto a = do { 1; 2 };   // ok, a is an int
auto b = do { 1; 2; };  // ill-formed, b would be void (unless Regular Void
        is adopted)
```

But this would mean that our original example would work, just for a very different reason (rather than being `void` expressions due to the lack of `do_return`, they become `void` expressions due to not having a final expression):

```
x match {
    0 => do { cout << "got zero"; };
    1 => do { cout << "got one"; };
    _ => do { cout << "don't care"; };
}
```

Ultimately, we feel that the simplicity of the proposed design and its consistency and uniformity with other parts of the language outweigh the added verbosity in the simple (though typical) cases.

§ 3.9 What About Reflection?

A question that often comes up, for any language feature: if we had reflection and, in particular, code injection: would we need this facility?

The answer is not only yes, but reflection is a good motivating use-case for this facility. Because the language does not have any kind of block expression today, adding support for one would increase the amount of ways that code injection could work.

One example might be, again, the control flow operator proposal in [P2561R2]. If reflection allows me to write a hygienic macro that does code injection, perhaps we could write a library such that `try_(E)` would inject an expression that would evaluate in the way that that paper proposes. But in order to do such a thing, we would need to be able to have a block expression to inject. This paper provides such a block expression.

§ 3.10 Where can `do` expressions appear

gcc's statement-expressions are not usable in all expression contexts. Trying to use them at namespace-scope, or in a default member initializer, etc, fails:

```
int i = ({      // error: statement-expressions are not allowed outside
         functions
    int j = 2;  //        nor in template-argument lists
    j;
});
```

In such contexts, there is a much smaller difference than a statement-expression and an immediately invoked lambda since you don't have any other interesting control flow that you can do - the expression either yields a value or the program terminates.

But if we're going to add a new language feature, it seems better to allow it to be used in all expression contexts - we would just have to say what happens in this case. Especially since if we're adding a feature to subsume immediately invoked lambdas, it would be preferable to subsume *all* immediately invoked lambdas, not just some or most.

We can think of a `do` expression as simply behaving like an immediately invoked lambda in such contexts. Not in the sense of allowing `return` statements (there's still no enclosing function to return out of), but the sense that any local variables declared would exist in a function stack. But this is probably more of a compiler implementation detail rather than a language design detail.

In short: `do` expressions should be usable in any expression context.

§ 4 Wording

This wording is quite incomplete, but is intended at this point to simply be a sketch to help understand the contour of the proposal.

Add to 7.5 [expr.prim]:

```
primary-expression:
  literal
  this
  ( expression )
  id-expression
  lambda-expression
  fold-expression
  requires-expression
+ do-expression
```

Add a new clause [expr.prim.do]:

1 A *do-expression* provides a way to combine multiple statements into a single expression without introducing a new function scope.

```
do-expression:
  do trailing-return-type_opt compound-statement
```

2 The *compound-statement* of a *do-expression* is a control-flow-limited statement ([stmt.label]).

Change 8.3 [stmt.expr] to disambugate a `do` expression from a `do-while` loop:

1 Expression statements have the form

```
expression-statement:
  expression_opt ;
```

The expression is a *discarded-value* expression. All side effects from an expression statement are completed before the next statement is executed. An expression statement with the expression missing is called a *null statement*. The expression shall not be a *do-expression*.

[Note 1: Most statements are expression statements — usually assignments or function calls. A null statement is useful to supply a null body to an iteration statement such as a while statement ([stmt.while]). — end note]

Add to 8.7.1 [stmt.jump.general]:

1 Jump statements unconditionally transfer control.

```
jump-statement:
  break ;
  continue ;
  return expr-or-braced-init-list_opt ;
+ do_return expr-or-braced-init-list_opt ;
  coroutine-return-statement
  goto identifier ;
```

Add a new clause introducing a `do_return` statement after 8.7.4 [stmt.return]:

> 1 The `do` expression's value is produced by the `do_return` statement.
>
> 2 A `do_return` statement shall appear only within the *`compound-statement`* of a `do` expression.

§ 5 References

[P1371R3] Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. 2020-09-15. Pattern Matching.
https://wg21.link/p1371r3

[P2552R2] Timur Doumler. 2023-05-19. On the ignorability of standard attributes.
https://wg21.link/p2552r2

[P2561R2] Barry Revzin. 2023-05-18. A control flow operator.
https://wg21.link/p2561r2

[P2688R0] Michael Park. 2022-10-16. Pattern Matching Discussion for Kona 2022.
https://wg21.link/p2688r0

[P2806R0] Barry Revzin, Bruno Cardoso Lopez, Zach Laine, Michael Park. 2023-02-14. do expressions.
https://wg21.link/p2806r0

[P2806R1] Barry Revzin, Bruno Cardoso Lopez, Zach Laine, Michael Park. 2023-03-12. do expressions.
https://wg21.link/p2806r1